

# Choose Your Own Project: Handwritten Character Recognition

edX HarvardX: PH125.9x - Data Science: Capstone

Azamat Kurbanaev

2026-10-09

# Contents

|                                                                                                                 |           |
|-----------------------------------------------------------------------------------------------------------------|-----------|
| <b>Preface</b>                                                                                                  | <b>2</b>  |
| <b>Acknowledgements</b>                                                                                         | <b>3</b>  |
| <b>1 Overview and Executive Summary</b>                                                                         | <b>4</b>  |
| 1.1 Project Description . . . . .                                                                               | 4         |
| 1.2 Environment Setup . . . . .                                                                                 | 6         |
| 1.2.1 Batch Execution & Cleanup Scripts . . . . .                                                               | 6         |
| 1.3 Running the Project . . . . .                                                                               | 7         |
| 1.4 Datasets Overview . . . . .                                                                                 | 7         |
| <b>2 Methods and Analysis</b>                                                                                   | <b>9</b>  |
| 2.1 k-Nearest Neighbors + Principal Component Analysis<br>(kNN+PCA) Multiclass Classifier (MCC) Model . . . . . | 9         |
| 2.2 Random Forest (RF) MCC Model . . . . .                                                                      | 9         |
| 2.3 Dense Neural Network-Based Basic Multiclass Classifier (DNNB<br>MCC) Model . . . . .                        | 9         |
| 2.4 Tuned DNN-Based (TDNN) MCC Model . . . . .                                                                  | 9         |
| 2.5 Convolutional Neural Network-Based Basic (CNNB) MCC Model                                                   | 9         |
| 2.6 Tuned CNN-Based (TCNN) MCC Model . . . . .                                                                  | 9         |
| <b>3 Results</b>                                                                                                | <b>10</b> |
| <b>4 Conclusion</b>                                                                                             | <b>11</b> |

# Preface

This project is the second assignment of the ‘Data Science: Capstone’ course (PH125.9x), offered by edX HarvardX. The project aims to build and compare different Handwritten Character Recognition (HCR) models, applying different Machine Learning (ML) techniques, and using a publicly available dataset for training and validation.

# Acknowledgements

I would like to extend my deepest gratitude to Professor **Rafael A. Irizarry** for designing and delivering the comprehensive courses within the ***Professional Certificate Program in Data Science***. Prof. **Irizarry**'s instruction has provided me with the essential theoretical foundations and practical toolkit required for modern data analysis.

I am equally grateful for his textbook, *Introduction to Data Science*, ([part 1](#)[1] and [part 2](#)[2]). This text served as an indispensable guide throughout my coursework, offering clear explanations and robust examples that bridged the gap between theory and execution. Prof. **Irizarry**'s dedication to creating clear, accessible educational material has profoundly enhanced my learning experience and will undoubtedly continue to serve as a key reference in my future endeavors.

# Chapter 1

## Overview and Executive Summary

As noted in the article [Handwritten Character Recognition: A Comprehensive Survey](#), Handwritten Character Recognition (HCR) plays a vital role in numerous domains, including document digitization, automatic text recognition, signature verification, and postal automation. Over the years, HCR has advanced significantly, driven by the growing demand for accurate, efficient recognition systems [3].

### 1.1 Project Description



All the source files for *this Project* are available in the *Project's* [GitHub repository](#)[4].

In this project, we will develop and explore several HCR models, based on popular ML algorithms that convert handwritten alphanumeric symbols into a digital, machine-readable form.

We will start with [Shallow Learning \(SL\)](#) models, using [k-Nearest Neighbors \(kNN\)](#) and [Random Forest \(RF\)](#) methods. These models development and evaluation are described in detail below in Sections [k-Nearest Neighbors + Principal Component Analysis \(kNN+PCA\)](#) [Multiclass Classifier \(MCC\) Model](#) and [Random Forest \(RF\) MCC Model](#) of *this Report*, respectively.

Next, we will try [Deep Learning \(DL\)](#) models, including [Dense Neural Network \(DNN\)](#) and, finally, [Convolutional Neural Network \(CNN\)](#) models, as recent re-

search shows that DL performs better at image recognition — the task addressed by *this Project*.

A more detailed explanation of *Deep Learning* versus *Shallow Learning* can be found in the chapter [Machine Learning Methods from Shallow Learning to Deep Learning](#) of the book [Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions](#) by editors Ömer Faruk Ertuğrul, Josep M. Guerrero, and Musa Yilmaz[5].

The ***DNN-Based*** models are as follows:

- ***DNN-Based Basic MCC Model:*** described in Section [Dense Neural Network-Based Basic Multiclass Classifier \(DNNB MCC\) Model](#);
- ***Tuned DNN-Based MCC Model:*** described in Section [Tuned DNN-Based \(TDNN\) MCC Model](#).

The last models we will build and explore, the ***CNN-Based*** models, are as follows:

- ***CNN-Based Basic MCC Model:*** described in Section [Convolutional Neural Network-Based Basic \(CNNB\) MCC Model](#);
- ***Tuned CNN-Based MCC Model:*** described in Section [Tuned CNN-Based \(TCNN\) MCC Model](#).

Finally, we will conduct a final test of the best model we have developed, using a specially designated dataset, and see what we will achieve.

## 1.2 Environment Setup



The environment setup instructions are provided in the [Prerequisites & Setup](#) section of the [README](#) document on [GitHub](#).

The environment setup configures an R-based machine learning environment with *Python* integration, required to build the *Deep Learning* models.

The installation follows a *4-step* process by sequentially running the following scripts:

- [1.install-reticulate&miniconda.R](#):
  1. *Installs [reticulate](#)*: Configures the primary R package, the [R Interface to Python](#);
  2. *Installs [Miniconda](#)* to manage Python virtual environments cleanly without conflicting with system Python installations.
- [2.install-tensorflow.R](#):
  1. **Installs the [R TensorFlow interface](#)**: Loads the TensorFlow R bindings;
  2. **Configures Backend Engine**: Uses reticulate to set up a dedicated *Python virtual/conda environment* and [installs the core C++/Python TensorFlow package](#) inside it.
- [3.install-keras3.R](#):
  1. **Installs the Keras3 package**: Installs the [R Interface for Keras3](#).
  2. **Binds High-Level Neural Network APIs**: Integrates Keras on top of the backend installed in [step 2](#) to enable building and training *Deep Learning* models in R.
- [4.install-kerastuner.R](#): Installs *Hyperparameter Tuning Tools*, including [R interface to Keras Tuner](#). Configures the [KerasTuner](#) in R to automate [Hyperparameter Optimization](#) (*search space, trial execution, and model evaluation*).

### 1.2.1 Batch Execution & Cleanup Scripts

As stated in the [Prerequisites & Setup](#) section of the [README](#) document on [GitHub](#), on *Windows OS*, all 4 steps of the setup process described above can be completed automatically by launching the [\\_\\_setup-env.cmd](#) *Windows command script* in *PowerShell* or *Command Prompt*.

The script [\\_\\_uninstall-all.R](#) cleans up the environment by wiping installed packages, *Conda environments*, and *Python bindings*.

## 1.3 Running the Project

Once the system environment is fully configured, if you have enough time to execute all the scripts and automatically generate *this report* upon their completion in a single session, the easiest approach is to run the `__run-all.R` script.

This script serves as the starting point for launching the project execution pipeline and subsequently generating a PDF report. It coordinates the process by sequentially executing two main R scripts, verifies the existence of the script files, and outputs detailed execution logs to both the console and log files.

The first script it launches, `index.R`, is the entry point to the project's execution pipeline and the main script that runs all the other execution scripts sequentially.

The second (and last) script it runs, `__rmd.render.R`, renders *this Report*.



Some scripts, being launched for the first time, can run for very long hours. For example, the longest run on the author's laptop was the first run of the `2.cnn-mcc.lr-tuner.R` script and took about 17 hours. Subsequent runs of these long-running scripts will be much faster thanks to a caching mechanism that checks for the existence of the pre-computed results before running time-consuming training or evaluation code and loads the results from the cache files (if any). Besides, any of these scripts can be launched separately, or the execution can be interrupted at any time and resumed later from the point of interruption.



If the interrupted execution resumes in another R session, the `Setup` section (with all its subsections ending at the beginning of the `kNN+PCA MCC Model` section) of the `index.R` script must be run first at the very beginning of the session.

## 1.4 Datasets Overview

The project uses the `HandWritten_Character Dataset` downloaded from the `Kaggle` platform.

As described on the `landing web-page`, the *context* of the dataset is as follows:

1. Used *EMNIST* data for Alphabets and Digits.
2. Transformed the Data using some image processing techniques and convert it to 32,32 pixel black and white images.
3. Created the Data set for special character ( @, #, \$, & )

4. Merged the Categories to avoid the misclassification
  5. Total 39 Categories in Train and Validation set
- 

using a [Kaggle Dataset](#).

## Chapter 2

# Methods and Analysis

- 2.1 k-Nearest Neighbors + Principal Component Analysis (kNN+PCA) Multiclass Classifier (MCC) Model
- 2.2 Random Forest (RF) MCC Model
- 2.3 Dense Neural Network-Based Basic Multiclass Classifier (DNNB MCC) Model
- 2.4 Tuned DNN-Based (TDNN) MCC Model
- 2.5 Convolutional Neural Network-Based Basic (CNNB) MCC Model
- 2.6 Tuned CNN-Based (TCNN) MCC Model

## Chapter 3

# Results

**Chapter 4**

**Conclusion**

# Appendix A: Support Functions

- [1] Rafael A. Irizarry. *Introduction to Data Science, Part I. Data Wrangling and Visualization with R*. Oct. 24, 2019. URL: <https://rafalab.dfci.harvard.edu/dsbook-part-1/> (cit. on p. 3).
- [2] Rafael A. Irizarry. *Introduction to Data Science, Part II. Statistics and Prediction Algorithms Through Case Studies*. Dec. 27, 2024. URL: <https://rafalab.dfci.harvard.edu/dsbook-part-2/> (visited on 02/18/2025) (cit. on p. 3).
- [3] Ajay Agarwal, Sartaj Ahmad, and Minakshi. “Handwritten Character Recognition: A Comprehensive Survey”. In: *2023 Second International Conference on Informatics (ICI)*. 2023, pp. 1–6. DOI: [10.1109/ICI60088.2023.10420981](https://doi.org/10.1109/ICI60088.2023.10420981). URL: <https://ieeexplore.ieee.org/document/10420981> (cit. on p. 4).
- [4] Azamat Kurbanov. *edX Data Science: Capstone, Choose Your Own. Handwritten Character Recognition Multiclass Classifier*. Version 1.0.0. Oct. 5, 2026. URL: <https://github.com/AzKurban-edX-DS/edx.capstone.choose-your-own> (cit. on p. 4).
- [5] Tahir Cetin Akinci, Oguzhan Topsakal, and M. Ilhan Akbas. “Machine Learning Methods from Shallow Learning to Deep Learning”. In: *Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions*. Ed. by Ömer Faruk Ertuğrul, Josep M. Guerrero, and Musa Yilmaz. Cham: Springer Nature Switzerland, 2024, pp. 1–28. ISBN: 978-3-031-69499-8. DOI: [10.1007/978-3-031-69499-8\\_1](https://doi.org/10.1007/978-3-031-69499-8_1). URL: [https://doi.org/10.1007/978-3-031-69499-8\\_1](https://doi.org/10.1007/978-3-031-69499-8_1) (cit. on p. 5).